

From Packets to Verdict: How Live Detection Works

A stage-by-stage walkthrough of the detection pipeline — how the network card sniffs packets in real time, how packets are grouped into flows, how a flow becomes 70 numbers, and how those numbers pass through 534 decision trees to a Benign-or-attack verdict.

THE PIPELINE AT A GLANCE

Six stages, one direction, packets in → alerts out

1	Sniff	NIC in promiscuous mode; libpcap copies every frame off the wire	raw packets
2	Assemble flows	group packets by 5-tuple; track each connection until it closes or times out	connections
3	Extract features	compute 80 statistics per flow (counts, sizes, timing, TCP flags)	one CSV row / flow
4	Align columns	rename to the model's names, drop 8 all-zero, order to the exact 70	70-number vector
5	Model inference	run the vector through 534 trees; sum leaves; sigmoid to a probability	p(attack) in 0..1
6	Decide & alert	p >= 0.5 -> SSH-Bruteforce; aggregate per source IP	verdict + alert

Data changes shape at every stage: a burst of packets becomes a handful of flows, each flow becomes one row of numbers, each row becomes one probability, and probabilities become alerts.

STAGE 1 — LIVE PACKET SNIFFING

Getting a copy of every frame on the wire

A network card normally ignores frames not addressed to it. Sniffing puts the interface in **promiscuous mode**, so it accepts every frame it sees, and the kernel's capture library **libpcap** hands a copy to user space. On the isolated host-only network, the victim VM sees the Kali VM's SSH packets, so capturing on the victim's interface records the whole attack.

Each captured packet carries a timestamp and its headers: the IP header (source and destination IP), the TCP header (source and destination ports, sequence numbers, and flag bits such as SYN, ACK, FIN, RST, PSH, URG), and the payload length. The detector never inspects payload contents — only these header facts and their timing.

```
# live: read frames straight off the interface
sudo cicflowmeter -i enp0s3 -c live_flows.csv
# or record first, analyse later:
sudo tcpdump -i enp0s3 -w attack.pcap
```

Live mode analyses frames as they arrive; offline mode replays a saved .pcap. The pipeline downstream is identical — only the packet source differs.

STAGE 2 — ASSEMBLING FLOWS

Turning a stream of packets into connections

A **flow** is one conversation. Packets are grouped by their **5-tuple** — source IP, source port, destination IP, destination port, and protocol. All packets sharing that key belong to the same flow. The first packet fixes the **forward** direction (client→server); replies are **backward**. The tool keeps a live table of open flows and updates each as new packets arrive.

A flow ends, and its row is emitted, when either:

- the TCP connection closes (a FIN or RST is seen), or
- it hits the **120-second flow timeout** (the same 120 s cap the cleaning step enforced), or
- an **activity timeout** splits a long-idle connection into active and idle segments.

One SSH login attempt is one short TCP connection — so a brute-force run of hundreds of attempts produces hundreds of short flows, each from the same source IP to port 22. That pattern is what the model ultimately keys on.

STAGE 3 — FEATURE EXTRACTION

Each flow becomes 80 numbers

When a flow ends, CICFlowMeter computes ~80 statistics describing it — the same features the model was trained on. They fall into a few families:

Feature family	Examples
Identity	Dst Port, Protocol
Packet & byte counts	Tot Fwd/Bwd Pkts, TotLen Fwd/Bwd Pkts, Subflow, Fwd Act Data Pkts
Packet-size stats	Fwd/Bwd Pkt Len max/min/mean/std, Pkt Len min/max/mean/std/var, Pkt Size Avg
Timing (inter-arrival)	Flow Duration, Flow/Fwd/Bwd IAT mean/std/max/min, Active & Idle
Rates	Flow Byts/s, Flow Pkts/s, Fwd/Bwd Pkts/s
TCP flags	FIN/SYN/RST/PSH/ACK/URG/CWE/ECE counts, Fwd/Bwd PSH & URG flags
Headers & window	Fwd/Bwd Header Len, Init Fwd/Bwd Win Byts, Fwd Seg Size Min

The result is **one CSV row per flow**. A 2-minute brute-force burst might yield a few hundred rows; a normal SSH session yields one longer row with very different values.

STAGE 4 — COLUMN ALIGNMENT

Matching the flow row to the model's exact inputs

The model expects **70 features, by name, in a fixed order** (stored in `model_features.json`). Two adjustments turn a CICFlowMeter row into that vector:

- **Rename** — CICFlowMeter's headers differ from the training names ("Total Fwd Packet" → "Tot Fwd Pkts"), so a rename map aligns them.
- **Select & order** — drop the identifier and Label columns and the 8 all-zero columns, then arrange the remaining 70 in the exact training order. XGBoost matches inputs by position, so order matters.

```
feats = json.load(open('model_features.json'))['features'] # 70 names, in order
df = df.rename(columns=CIC_TO_TRAIN) # align header names
X = df[feats].astype('float32') # select + order + XGBoost precision
```

STAGE 5 — MODEL INFERENCE

How 70 numbers become one probability

The classifier is 534 small decision trees. For a flow's 70-number vector x :

- **Each tree routes x to a leaf.** At every node the tree asks one yes/no question about one feature (e.g. "Fwd Act Data Pkts < 3?"); the answer picks the next branch until x reaches a leaf holding a small positive or negative number.
- **Sum the leaves.** The 534 leaf values (plus a base score) add up to one number, the log-odds (the "raw margin").
- **Squash to a probability.** The sigmoid function maps that number to p between 0 and 1 — the model's estimated probability the flow is an SSH brute force.

```
logit = base + tree_1(x) + tree_2(x) + ... + tree_534(x)      # sum of leaf values
p      = 1 / (1 + e^(-logit))                                # sigmoid -> 0..1

# illustrative single flow (one Patator login attempt):
x: Dst Port=22, Fwd Act Data Pkts=1, Fwd Header Len=32, SYN=1, ...
logit ~ +6.3         ->      p = 0.998         ->      class 1 (attack)
```

scale_pos_weight only influenced **training**; at prediction time the model just sums trees. The 0.5 threshold converts p to a label: $p \geq 0.5 \rightarrow$ SSH-Bruteforce (1), else Benign (0). You can raise the threshold to trade recall for fewer false alarms.

STAGE 6 — DECISION & ALERTING

From per-flow labels to one actionable alert

Each flow now has a label and a probability. On their own, single flows are noisy, so the detector **aggregates by source IP**: it counts how many flows from each IP were classified as attack over a short window and raises one alert when that count crosses a threshold. This turns hundreds of individual attack flows into a single, clear signal.

```
[flow] 192.168.56.10:51xxx -> 192.168.56.20:22 p=0.998 SSH-Bruteforce
[flow] 192.168.56.10:51xxx -> 192.168.56.20:22 p=0.997 SSH-Bruteforce
... (hundreds more) ...
>> ALERT 192.168.56.10 -> port 22 | 214 brute-force flows in 90 s
```

END-TO-END TIMING

When each detection appears

Detection is per flow, not per packet, so a verdict for a connection appears only after that connection ends. For SSH brute force, each login attempt is a short-lived connection, so flows close in about a second and verdicts stream in continuously during the attack. A connection that stays open is only scored when it closes or hits the 120-second timeout — the deliberate cost of flow-level detection.

Time	What happens
$t = 0$ s	Patator opens the first SSH connection; packets start arriving at the victim
$t \sim 0.5$ s	that attempt fails and the connection closes; the flow is finalised
$t \sim 0.6$ s	CICFlowMeter emits the flow row; column alignment builds the 70-vector
$t \sim 0.6$ s	the model scores it $p = 0.99 \rightarrow$ SSH-Bruteforce
$t \sim 2-5$ s	dozens of attack flows counted from one IP $\rightarrow$ ALERT raised
continuous	new attempts keep producing flows and verdicts until Patator stops

WHY AN SSH BRUTE FORCE STANDS OUT

What the numbers actually show

A brute force is many short, near-identical login attempts. In flow features that means: very few forward data packets per flow (one attempt each), small and uniform packet and header lengths, a SYN and quick teardown, and a rapid, regular arrival pattern from one source to port 22. A genuine SSH session is the opposite — one longer flow with many packets, varied sizes, and interactive timing. The model's top features (Fwd Act Data Pkts, Fwd Header Len, Dst Port) are exactly these signals.

Pairs with the lab test procedure ([ssh_bruteforce_lab_test_procedure.pdf](#)). Compiled 2026-09-28.